

INTER IIT TECH MEET 15.0 *PREPATHON PS*

ALGORITHMS

DCs: Parth Dabas (8580493331)

Algo Prepathon — Problem Statement

Flexible Job-Shop Scheduling (FJSP)

1. OVERVIEW

You are given the set of jobs J and the set of machines M .

- Each job is an ordered sequence of operations.
- Each operation can run on one or more eligible machines.
- Processing time depends on which machine is chosen.

Goal: build a valid schedule that minimizes the makespan

$$C_{\max} = \max_j C_j \quad (1)$$

But this Prepathon is not only about writing a scheduler. The real objective is the full pipeline:

Pipeline

Generate → Validate → Solve → Stress Test → Analyze → Improve

2. WHAT AN INSTANCE MUST CONTAIN

An FJSP instance is fully defined by:

Element	Description
$M = \{1..m\}$	Set of machines, $m \geq 1$
$J = \{1..n\}$	Set of jobs, $n \geq 1$
k_j	Number of operations in job j , $k_j \geq 1$ (may differ per job)
$O(j, 1..k_j)$	Ordered operation sequence of job j (a strict chain, no branching)
$E(j, k) \subseteq M$	Eligible machine set of operation $O(j, k)$, non-empty
$p(j, k, m)$	Processing time of $O(j, k)$ on machine $m \in E(j, k)$, integer and > 0

Rules an instance must obey to be well-formed:

- Every operation has **at least one** eligible machine ($|E(j, k)| \geq 1$).
- A processing time is defined for **every** machine in $E(j, k)$, and for none outside it.
- All processing times are strictly positive and finite. No zero-duration operations.
- Machine and job IDs are contiguous and within range; no duplicate operation IDs within a job.
- Operations within a job form one linear precedence chain in the given order.
- Total operations = $\sum_j k_j$. The instance must be feasible by construction — a valid schedule always exists.

Example of a single operation:

```
Operation O(2,3):
Machine 1 → 8
Machine 3 → 5
Machine 4 → 11
```

The scheduler must pick **exactly one** of these machines.

Your instance file/format must store all of the above plus the metadata needed to reproduce it (seed, generator parameters, instance class).

3. SCHEDULING CONSTRAINTS

For a schedule, let $S(j, k)$ be the start time and $C(j, k)$ the completion time of operation $O(j, k)$. A schedule is valid only if **all** of the following hold:

1. **Machine assignment** — every operation is assigned to exactly one machine, and that machine must be in $E(j, k)$. Assigning to an ineligible machine, to more than one machine, or leaving an operation unassigned is invalid.
2. **Duration correctness** — $C(j, k) = S(j, k) + p(j, k, m)$ for the assigned machine m . The duration must match the instance's processing time exactly.
3. **Job precedence** — $S(j, k + 1) \geq C(j, k)$ for all k . An operation cannot start before its predecessor in the same job finishes. Two operations of the same job may never overlap in time, even on different machines.
4. **Machine non-overlap** — if operations A and B are on the same machine, then $C(A) \leq S(B)$ or $C(B) \leq S(A)$. No machine processes two operations simultaneously.
5. **Non-preemption** — once an operation starts it runs to completion; it cannot be paused, split, or moved to another machine mid-execution.
6. **Non-negative time** — $S(j, k) \geq 0$ for all operations.
7. **Completeness** — every operation of every job appears exactly once in the schedule. No duplicates, no omissions, no unknown job/operation/machine IDs.

Derived quantities: $C_j = C(j, k_j)$ is the completion time of job j , and $C_{\max} = \max_j C_j$ (Eq. Eq. 1) is the makespan. The objective is $\min C_{\max}$.

Version 1 Assumptions

All jobs are available at $t = 0$; no release dates, due dates, setup times, transport times, or machine breakdowns; deterministic machines and processing times; unlimited buffering between operations; jobs may wait; machines may sit idle.

Scope

Do **not** add extra features (due dates, breakdowns, preemption, stochastic times, dynamic arrivals) to the base problem. They are optional extensions only after the base works.

4. YOUR TASK

Part A — Random Instance Generator

You must write the logic of the random data generator yourself — not just call a library or dump uniform random numbers into a file. The generator is one of the most important parts of this challenge and is graded on its design.

Required parameters (at minimum):

```
number_of_jobs, number_of_machines, operations_per_job,
machine_flexibility, processing_time_range,
processing_time_variance, bottleneck_probability, seed
```

Must be able to produce: average, easy, hard, extreme, bottleneck-heavy, high-flexibility, low-flexibility, unbalanced, high-variance, and varying job/machine ratio instances.

Every generated instance must satisfy all the well-formedness rules in Section 2 **by construction** — the generator can never emit an invalid instance.

You must write up the generator's logic explicitly:

- How the eligible machine set of each operation is sampled, and how flexibility is controlled.

- Which probability distributions produce processing times, and how variance is tuned.
- How bottlenecks, machine advantage, and imbalance are deliberately injected.
- How each named instance class (low flexibility, bottleneck, high variance, etc.) maps to concrete parameter settings.
- Why the construction guarantees validity.

Expose a **seed** so that any instance can be regenerated exactly.

Part B — Schedule Validation Environment

Build an **independent** evaluator that checks a proposed schedule. It must not trust the solver.

Minimum schedule representation per operation:

```
job_id, operation_id, machine_id, start_time, completion_time
```

It must catch: ineligible machine, multiple/duplicate/missing operations, negative start time, wrong processing or completion time, precedence violation, machine overlap, invalid job/operation/machine IDs, malformed or incomplete schedules.

Output **VALID** only if every check passes; otherwise **INVALID** with clear diagnostics, e.g.

```
Error: Machine 5 contains overlapping operations:
Job 1 Op 3: [10, 18]
Job 6 Op 1: [15, 22]
```

It must also reconstruct machine schedules, job schedules, job completion times and the overall makespan.

Part C — Implement a Scheduling Algorithm

Implement **at least one** algorithm. Any approach is allowed: greedy/dispatching rules, local search, simulated annealing, tabu search, GA, PSO, ACO, MILP, CP-SAT, LNS, RL, GNNs, or hybrids — your own or from a paper. Document: the core idea, representation, search space, objective, operators/neighborhood, hyperparameters, complexity, and what you changed from the original if you followed a paper.

If you use a paper, understand it — what it solves, its assumptions, what it was tested on, its weaknesses, and whether those weaknesses show up in your instances.

Part D — Experiments and Failure Analysis

Test on multiple instance classes, not just average random cases:

Instance Type	Purpose
Average	Normal expected behavior
Low / high flexibility	Routing constraints vs. machine-selection decisions
Bottleneck	Machine contention
Balanced	Normal load distribution
High variance	Very different operation lengths
Machine advantage	Exploiting specialization
Large / small	Scalability vs. strong correctness checks
Extreme	Discover failure modes

Report per experiment: instance characteristics, feasibility, makespan, runtime, #jobs, #machines, #operations. Where possible also machine utilization, critical path, lower-bound gap, convergence.

Hand-built edge cases (don't rely on random generation alone): single machine; one job; many jobs with one bottleneck machine; extreme time gaps (e.g. $M_1 \rightarrow 1$ vs $M_2 \rightarrow 10000$); machine advantage; near-total flexibility; near-zero flexibility; identical processing times; long critical chains; many very short operations.

Failure analysis is the most important part. For each failure mode document:

1. **Observation** — what happened.
2. **Evidence** — experiments showing it (e.g. makespan vs. bottleneck probability).
3. **Hypothesis** — why you think it happens.
4. **Structural explanation** — tie it to precedence, contention, flexibility, critical paths, assignment vs. sequencing decisions, search-space size.
5. **Proposed improvement** — an idea only; **no implementation required**.

Example of the reasoning expected:

Example Reasoning

High flexibility → larger search space → algorithm commits to machines too early → later operations become bottlenecks → makespan increases.

5. REPRODUCIBILITY

Record seed, generation parameters, instance, algorithm config and parameters, runtime, and result. Every experiment must be rerunnable from its seed. Never draw conclusions from a single random instance — use multiple seeds and report aggregates.

6. SUBMISSION

1. Create a **public GitHub repository** containing your complete solution (code, instances, results, and README/report).
2. Upload/push everything to that repo.
3. Submit the repo link via <https://forms.gle/LWQvHz7BFBT9GUB57> by **7th October**.

Suggested structure (optional):

```
fjsp/  
├── generator/      # instance_generator  
├── environment/   # validator, evaluator, metrics  
├── algorithms/    # your_algorithm  
├── experiments/   # average, edge_cases, extreme_cases  
├── analysis/      # results  
├── instances/  
├── results/  
└── README.md
```

Your report should cover: the generator (representation, distributions, parameters, seeds, why instances are valid), the validator (rules, error reporting, tests proving invalid schedules are rejected), the algorithm, the experiments, and the failure analysis.

7. EVALUATION PHILOSOPHY

You are **not** judged only on how low your makespan is. You are judged on your ability to reason about an optimization problem — from model to generator to validation to algorithm to experiments to failure cases to structural explanation to proposed improvement.

What Is Valued

"Our algorithm is excellent on low-flexibility instances but fails on high-flexibility bottleneck instances because its early machine assignments ignore downstream congestion."

is worth more than

"Our algorithm achieved makespan 381."

SUGGESTED FIRST MILESTONE

Get this pipeline working before optimizing anything:

Random Generator → Valid Instance → Candidate Schedule
→ Independent Validator → Job Completion Times → Makespan

A sophisticated algorithm on top of a broken generator or validator produces meaningless results.

8. SUBMISSION DEADLINE

Deadline & Submission

Deadline: 18 September 2026

Submission link: <https://forms.gle/LWQvHz7BFBT9GUB57>